Evasion Defenses

A. M. Sadeghzadeh, Ph.D.

Sharif University of Technology
Computer Engineering Department (CE)
Trustworthy and Secure AI Lab

November 16, 2025

Today's Agenda

Defensive Distillation

Defensive Distillation

Accepted to the 37th IEEE Symposium on Security & Privacy, IEEE 2016. San Jose, CA.

Distillation as a Defense to Adversarial Perturbations against Deep Neural Networks

Nicolas Papernot*, Patrick McDaniel*, Xi Wu§, Somesh Jha§, and Ananthram Swami‡
*Department of Computer Science and Engineering, Penn State University
§Computer Sciences Department, University of Wisconsin-Madison
‡United States Army Research Laboratory, Adelphi, Maryland
{ngp5056,mcdaniel}@cse.psu.edu, {xiwu,jha}@cs.wisc.edu, ananthram.swami.civ@mail.mil

Defensive Distillation

2017 IEEE Symposium on Security and Privacy

Towards Evaluating the Robustness of Neural Networks

Nicholas Carlini David Wagner
University of California, Berkeley

Defensive Distillation

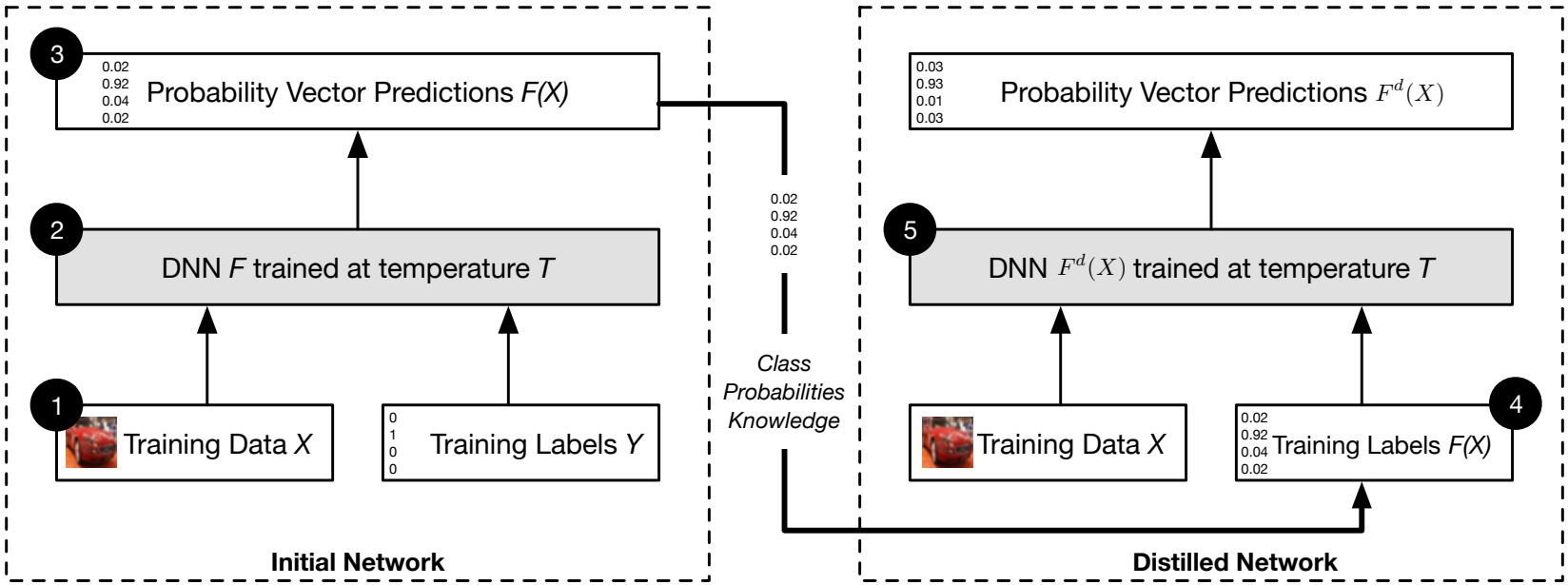
Distillation was initially proposed as an approach to **reduce a large model** (the teacher) down to a smaller distilled model [7].

Defensive Distillation

Distillation was initially proposed as an approach to **reduce a large model** (the teacher) down to a smaller distilled model [7].
Papernot et al. [9] use distillation in order to increase the robustness of a neural network, but with two significant changes.

1. Both the teacher model and the distilled model are **identical in size**.
2. Defensive distillation uses a large **distillation temperature** in the softmax function to force the distilled model to become more confident in its predictions.

$$\text{softmax}(x, T)_i = \frac{e^{x_i/T}}{\sum_j e^{x_j/T}}$$



Defensive Distillation

Defensive distillation proceeds in four steps:

1. **Train the teacher network**, by setting the temperature of the softmax to T during the training phase.

Defensive Distillation

Defensive distillation proceeds in four steps:

1. **Train the teacher network**, by setting the temperature of the softmax to T during the training phase.
2. **Compute soft labels** by apply the teacher network to each instance in the training set, again evaluating the softmax at temperature T.

Defensive Distillation

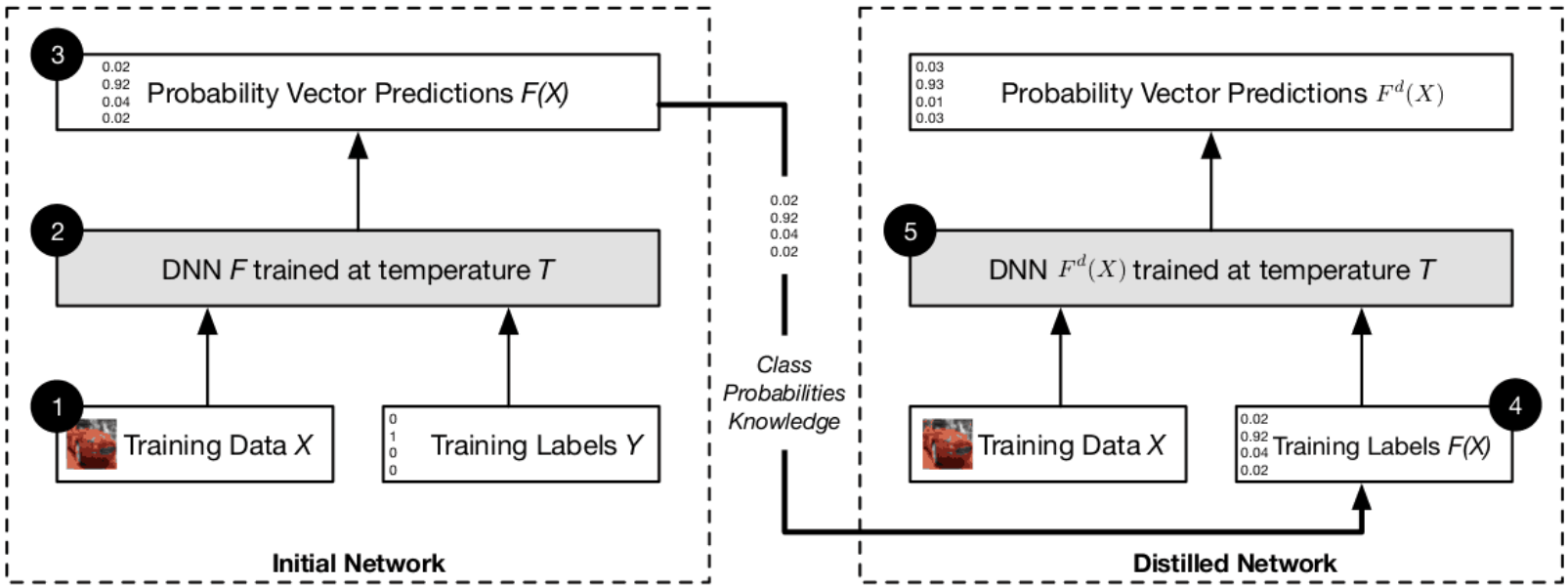
Defensive distillation proceeds in four steps:

1. **Train the teacher network**, by setting the temperature of the softmax to T during the training phase.
2. **Compute soft labels** by apply the teacher network to each instance in the training set, again evaluating the softmax at temperature T.
3. **Train the distilled network** (a network with the same shape as the teacher network) on the soft labels, using softmax at temperature T.

Defensive Distillation

Defensive distillation proceeds in four steps:

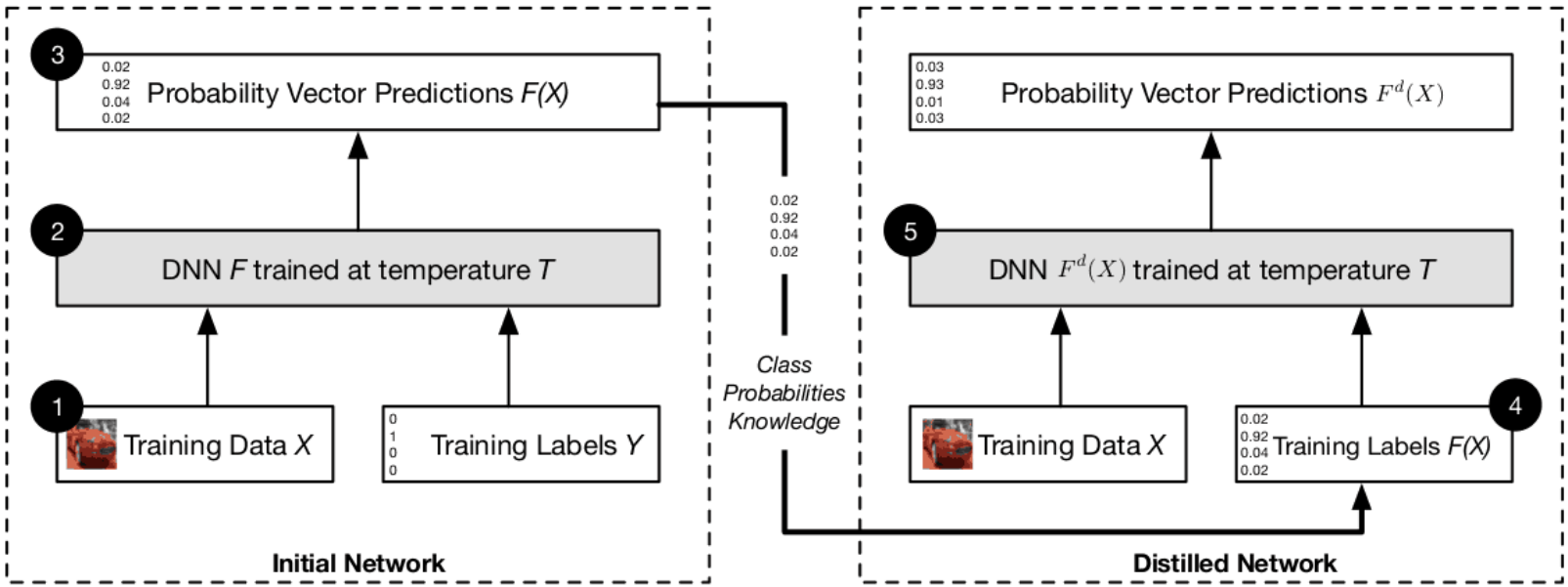
1. **Train the teacher network**, by setting the temperature of the softmax to T during the training phase.
2. **Compute soft labels** by apply the teacher network to each instance in the training set, again evaluating the softmax at temperature T.
3. **Train the distilled network** (a network with the same shape as the teacher network) on the soft labels, using softmax at temperature T.
4. Finally, when running the distilled network at test time to **classify new inputs**, use temperature 1.



Defensive Distillation

Defensive distillation proceeds in four steps:

1. **Train the teacher network**, by setting the temperature of the softmax to T during the training phase.
2. **Compute soft labels** by apply the teacher network to each instance in the training set, again evaluating the softmax at temperature T.
3. **Train the distilled network** (a network with the same shape as the teacher network) on the soft labels, using softmax at temperature T.
5. Finally, when running the distilled network at test time to **classify new inputs**, **use temperature 1**.



Softmax Derivative

Let $\hat{\boldsymbol{y}} = \mathit{softmax}(\boldsymbol{z})$ where $\boldsymbol{z}, \hat{\boldsymbol{y}} \in \mathbb{R}^3$ and $\boldsymbol{z} = [-2.85, 5.23, 0.28]^T$, we have

$$\hat{\boldsymbol{y}} = \mathit{softmax}(\begin{bmatrix} -2.85 \\ 5.23 \\ 0.28 \end{bmatrix}, T = 1) = \begin{bmatrix} 3.0739841436031E-4 \\ 0.99266117654418 \\ 0.0070314250414565 \end{bmatrix}$$

Softmax Derivative

Let $\hat{\boldsymbol{y}} = softmax(\boldsymbol{z})$ where $\boldsymbol{z}, \hat{\boldsymbol{y}} \in \mathbb{R}^3$ and $\boldsymbol{z} = [-2.85, 5.23, 0.28]^T$, we have

$$\hat{\boldsymbol{y}} = softmax(\begin{bmatrix} -2.85 \\ 5.23 \\ 0.28 \end{bmatrix}, T = 1) = \begin{bmatrix} 3.0739841436031E-4 \\ 0.99266117654418 \\ 0.0070314250414565 \end{bmatrix}$$

With $T = 100$, Logits $\boldsymbol{z}$ should become larger by a factor of T to $\hat{\boldsymbol{y}}$ does not change.

$$\hat{\boldsymbol{y}} = softmax(\begin{bmatrix} -285 \\ 523 \\ 28 \end{bmatrix}, T = 100) = \begin{bmatrix} 3.0739841436031E-4 \\ 0.99266117654418 \\ 0.0070314250414565 \end{bmatrix}$$

Softmax Derivative

Let $\hat{\boldsymbol{y}} = softmax(\boldsymbol{z})$ where $\boldsymbol{z}, \hat{\boldsymbol{y}} \in \mathbb{R}^3$ and $\boldsymbol{z} = [-2.85, 5.23, 0.28]^T$, we have

$$\hat{\boldsymbol{y}} = softmax(\begin{bmatrix} -2.85 \\ 5.23 \\ 0.28 \end{bmatrix}, T = 1) = \begin{bmatrix} 3.0739841436031E-4 \\ 0.99266117654418 \\ 0.0070314250414565 \end{bmatrix}$$

With $T = 100$, Logits $\boldsymbol{z}$ should become larger by a factor of T to $\hat{\boldsymbol{y}}$ does not change.

$$\hat{\boldsymbol{y}} = softmax(\begin{bmatrix} -285 \\ 523 \\ 28 \end{bmatrix}, T = 100) = \begin{bmatrix} 3.0739841436031E-4 \\ 0.99266117654418 \\ 0.0070314250414565 \end{bmatrix}$$

The distilled network uses $T = 1$ to classify new inputs in the test time. Since the network parameters are fixed, **logits *z* remain large**. Hence, the distilled network becomes overconfident, and the gradients of the softmax become significantly small. We have

$$\hat{\boldsymbol{y}} = softmax(\begin{bmatrix} -285 \\ 523 \\ 28 \end{bmatrix}, T = 1) = \begin{bmatrix} 1.2304348468251E-351 \\ 1 \\ 1.0573808917922E-215 \end{bmatrix} \approx \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

Softmax Derivative

Recall: Softmax Jacobian

Let $\hat{\boldsymbol{y}} = \mathit{softmax}(\boldsymbol{z})$ where $\boldsymbol{z}, \hat{\boldsymbol{y}} \in \mathbb{R}^3$, we have

$$\frac{\partial \hat{\boldsymbol{y}}}{\partial \boldsymbol{z}} = \begin{bmatrix} \frac{\partial \hat{y}_1}{\partial z_1} & \frac{\partial \hat{y}_1}{\partial z_2} & \frac{\partial \hat{y}_1}{\partial z_3} \\ \frac{\partial \hat{y}_2}{\partial z_1} & \frac{\partial \hat{y}_2}{\partial z_2} & \frac{\partial \hat{y}_2}{\partial z_3} \\ \frac{\partial \hat{y}_3}{\partial z_1} & \frac{\partial \hat{y}_3}{\partial z_2} & \frac{\partial \hat{y}_3}{\partial z_3} \end{bmatrix} = \begin{bmatrix} \hat{y}_1(1-\hat{y}_1) & -\hat{y}_2\hat{y}_1 & -\hat{y}_3\hat{y}_1 \\ -\hat{y}_1\hat{y}_2 & \hat{y}_2(1-\hat{y}_2) & -\hat{y}_3\hat{y}_2 \\ -\hat{y}_1\hat{y}_3 & -\hat{y}_2\hat{y}_3 & \hat{y}_3(1-\hat{y}_3) \end{bmatrix}$$

$$\approx \begin{bmatrix} 0(1-0) & -1\times 0 & -0\times 0 \\ -0\times 1 & 1(1-1) & -0\times 1 \\ -0\times 0 & -1\times 0 & 0(1-0) \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Defensive Distillation

- L-BFGS fail due to the fact that the gradient of the classifier is zero almost always, which prohibits the use of the standard objective function.
- Since $Z()$ is divided by T, the distilled network will learn to make the $Z()$ values T times larger than they otherwise would be.
- Experimentally, we verified this fact: the **mean value of the L1 norm** of $Z()$ (the logits) on the undistilled network is 5.8 with standard deviation 6.4; on the distilled network (with $T = 100$), the mean is 482 with standard deviation 457.

Defensive Distillation

- An alternate approach to fixing L-BFGS attack would be to set $T = 100$ at the test time

$$F'(x) = softmax(Z(x)/100)$$

  - This approach does not work for FGSM attack (why?).
- When **C&W attack** applies to defensively distilled networks, **distillation provides only marginal value**.
- The second break of distillation is through **transferring** attacks from a standard model to a defensively distilled model.

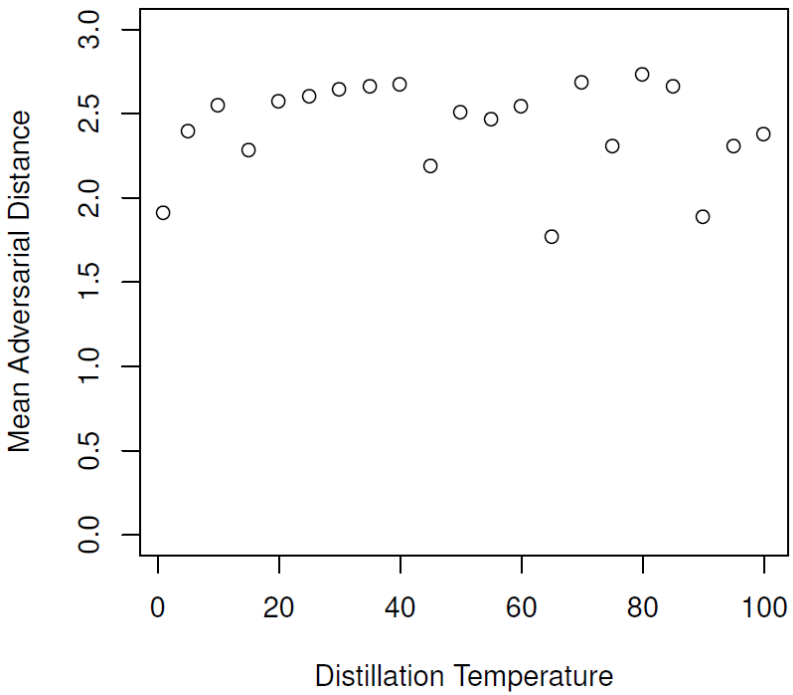
	Best Case				Average Case				Worst Case			
	MNIST		CIFAR		MNIST		CIFAR		MNIST		CIFAR	
	mean	prob	mean	prob	mean	prob	mean	prob	mean	prob	mean	prob
Our L_0	10	100%	7.4	100%	19	100%	15	100%	36	100%	29	100%
Our L_2	1.7	100%	0.36	100%	2.2	100%	0.60	100%	2.9	100%	0.92	100%
Our L_∞	0.14	100%	0.002	100%	0.18	100%	0.023	100%	0.25	100%	0.038	100%

TABLE VI
COMPARISON OF OUR ATTACKS WHEN APPLIED TO DEFENSIVELY DISTILLED NETWORKS. COMPARE TO TABLE IV FOR UNDISTILLED NETWORKS.

Temperature

In the original work, increasing the temperature was found to consistently reduce attack success rate.

The experiments on C&W attack clearly demonstrate the fact that increasing the distillation **temperature does not increase the robustness of the neural network**.



Obfuscated Gradients

Obfuscated Gradients

Obfuscated Gradients Give a False Sense of Security: Circumventing Defenses to Adversarial Examples

Anish Athalye [*1] **Nicholas Carlini** [*2] **David Wagner** [2]

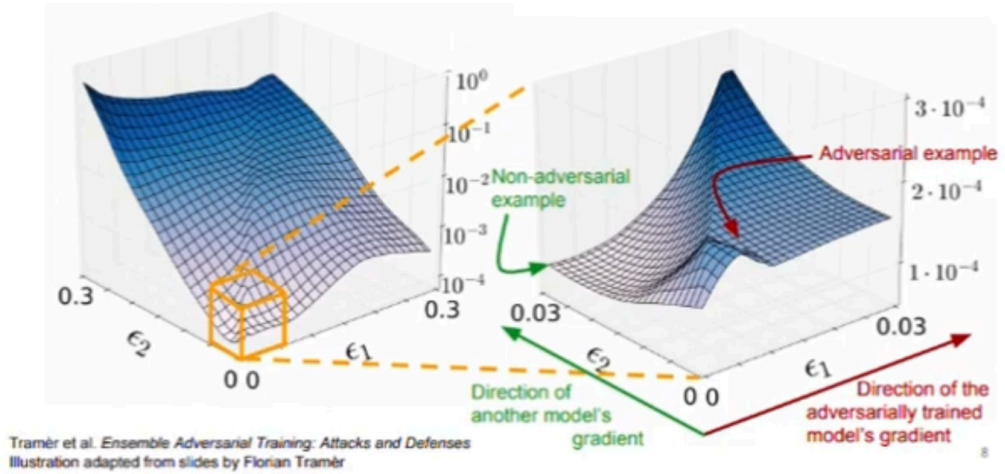
Abstract

- We identify **obfuscated gradients** as a phenomenon that leads to a **false sense of security** in defenses against adversarial examples.
  - Without a good gradient, where following the gradient does not successfully optimize the loss, iterative optimization-based methods cannot succeed.
- We describe **characteristic behaviors of defenses** exhibiting the effect, and for each of the three types of obfuscated gradients we discover, **we develop attack techniques to overcome it**.
- In a case study, examining noncertified white-box-secure defenses at **ICLR 2018**, we find obfuscated gradients are a common occurrence, with **7 of 9 defenses relying on obfuscated gradients**.

Gradient Masking

Definition:

a term used to describe an entire category of failed defense methods that work by trying to deny the attacker access to a useful gradient. Most adversarial example construction techniques use the gradient of the model to attack.



(a) Defended model (b) Substitute model

Notation

- We consider a neural network $f(.)$ used for classification where $f(x)_i$ represents the probability that image x corresponds to label i.
- We classify images, represented as $x \in [0,1]^{w.h.c}$ for a c-channel image of width w and height h.
- We use $f^j(.)$ to refer to layer j of the neural network, and $f^{1..j}(.)$ the composition of layers 1 through j.
- We denote the classification of the network as $c(x) = \underset{i}{\mathit{argmax}}\ f(x)_i$, and $c^*(x)$ denotes the true label.
- We use the ℓ_∞ and ℓ_2 distortion metrics to measure similarity.

Threat Models and Attack Methods

We consider defenses designed for the **white-box setting**, where the adversary has full access to the neural network classifier (architecture and weights) and defense, **but not test-time randomness (only the distribution)**.

We construct adversarial examples with iterative optimization-based methods.

- To generate ℓ_∞ bounded adversarial examples, we use **Projected Gradient Descent (PGD)**
- To generate ℓ_2 bounded adversarial examples, we use the Lagrangian relaxation of **Carlini & Wagner (C&W)**

Obfuscated Gradients

A defense is said to cause gradient masking if it **does not have useful gradients** for generating adversarial examples.

- we refer to the case where defenses are designed in such a way that the constructed defense necessarily causes gradient masking as obfuscated gradients.

Obfuscated Gradients

A defense is said to cause gradient masking if it **does not have useful gradients** for generating adversarial examples.

- we refer to the case where defenses are designed in such a way that the constructed defense necessarily causes gradient masking as obfuscated gradients.

We discover three ways in which defenses obfuscate gradients

Obfuscated Gradients

A defense is said to cause gradient masking if it **does not have useful gradients** for generating adversarial examples.

- we refer to the case where defenses are designed in such a way that the constructed defense necessarily causes gradient masking as obfuscated gradients.

We discover three ways in which defenses obfuscate gradients

1. **Shattered Gradients**
   Shattered Gradients are caused when a defense is nondifferentiable, introduces numeric instability, or otherwise causes a **gradient to be nonexistent or incorrect**.

Obfuscated Gradients

A defense is said to cause gradient masking if it **does not have useful gradients** for generating adversarial examples.

- we refer to the case where defenses are designed in such a way that the constructed defense necessarily causes gradient masking as obfuscated gradients.

We discover three ways in which defenses obfuscate gradients

1. **Shattered Gradients**
   Shattered Gradients are caused when a defense is nondifferentiable, introduces numeric instability, or otherwise causes a **gradient to be nonexistent or incorrect**.
2. **Stochastic Gradients**
   Stochastic Gradients are caused by **randomized defenses**, where either the network itself is randomized or the input is randomly transformed before being fed to the classifier, causing the gradients to become randomized.

Obfuscated Gradients

A defense is said to cause gradient masking if it **does not have useful gradients** for generating adversarial examples.

- we refer to the case where defenses are designed in such a way that the constructed defense necessarily causes gradient masking as obfuscated gradients.

We discover three ways in which defenses obfuscate gradients

1. **Shattered Gradients**
   Shattered Gradients are caused when a defense is nondifferentiable, introduces numeric instability, or otherwise causes a **gradient to be nonexistent or incorrect**.
2. **Stochastic Gradients**
   Stochastic Gradients are caused by **randomized defenses**, where either the network itself is randomized or the input is randomly transformed before being fed to the classifier, causing the gradients to become randomized.
3. **Exploding & Vanishing Gradients**
   Exploding & Vanishing Gradients are often caused by defenses that consist of **multiple iterations of neural network** evaluation, feeding the output of one computation as the input of the next. This type of computation, when unrolled, can be viewed as an **extremely deep neural network evaluation**, which can cause vanishing/exploding gradients.

Identifying Obfuscated & Masked Gradients

- **One-step attacks perform better than iterative attacks.**
  - Iterative optimization-based attacks are strictly stronger than single-step attacks.
  - If single-step methods give performance superior to iterative methods, it is likely that the iterative attack is becoming stuck in its optimization search at a local minimum.

Identifying Obfuscated & Masked Gradients

- **One-step attacks perform better than iterative attacks.**
  - Iterative optimization-based attacks are strictly stronger than single-step attacks.
  - If single-step methods give performance superior to iterative methods, it is likely that the iterative attack is becoming stuck in its optimization search at a local minimum.
- **Black-box attacks are better than white-box attacks.**
  - Attacks in the white-box setting should perform better.
  - If a defense is obfuscating gradients, then black-box attacks (which do not use the gradient) often perform better than white-box attacks.

Identifying Obfuscated & Masked Gradients

- **One-step attacks perform better than iterative attacks.**
  - Iterative optimization-based attacks are strictly stronger than single-step attacks.
  - If single-step methods give performance superior to iterative methods, it is likely that the iterative attack is becoming stuck in its optimization search at a local minimum.
- **Black-box attacks are better than white-box attacks.**
  - Attacks in the white-box setting should perform better.
  - If a defense is obfuscating gradients, then black-box attacks (which do not use the gradient) often perform better than white-box attacks.
- **Unbounded attacks do not reach 100% success.**
  - With unbounded distortion, any classifier should have 0% robustness to attack.

Identifying Obfuscated & Masked Gradients

- **One-step attacks perform better than iterative attacks.**
  - Iterative optimization-based attacks are strictly stronger than single-step attacks.
  - If single-step methods give performance superior to iterative methods, it is likely that the iterative attack is becoming stuck in its optimization search at a local minimum.
- **Black-box attacks are better than white-box attacks.**
  - Attacks in the white-box setting should perform better.
  - If a defense is obfuscating gradients, then black-box attacks (which do not use the gradient) often perform better than white-box attacks.
- **Unbounded attacks do not reach 100% success.**
  - With unbounded distortion, any classifier should have 0% robustness to attack.
- **Random sampling finds adversarial examples.**
  - Bruteforce random search (e.g., randomly sampling 10^5 or more points) within some ϵ-ball should not find adversarial examples when gradient-based attacks do not.

Identifying Obfuscated & Masked Gradients

- **One-step attacks perform better than iterative attacks.**
  - Iterative optimization-based attacks are strictly stronger than single-step attacks.
  - If single-step methods give performance superior to iterative methods, it is likely that the iterative attack is becoming stuck in its optimization search at a local minimum.
- **Black-box attacks are better than white-box attacks.**
  - Attacks in the white-box setting should perform better.
  - If a defense is obfuscating gradients, then black-box attacks (which do not use the gradient) often perform better than white-box attacks.
- **Unbounded attacks do not reach 100% success.**
  - With unbounded distortion, any classifier should have 0% robustness to attack.
- **Random sampling finds adversarial examples.**
  - Bruteforce random search (e.g., randomly sampling 10^5 or more points) within some ϵ-ball should not find adversarial examples when gradient-based attacks do not.
- **Increasing distortion bound does not increase success.**
  - A larger distortion bound should monotonically increase attack success rate.

Attack Techniques

There is a number of techniques to overcome obfuscated gradients

- **Backward Pass Differentiable Approximation (DBPA)** to overcome shattered gradients
- **Expectation over Transformation** to overcome stochastic gradients
- **Reparameterization** to overcome exploding & vanishing gradients

Backward Pass Differentiable Approximation (BPDA)

To attack defenses where gradients are not readily available, such as shattered gradients, we introduce a technique we call **Backward Pass Differentiable Approximation (BPDA)**.

Backward Pass Differentiable Approximation (BPDA)

To attack defenses where gradients are not readily available, such as shattered gradients, we introduce a technique we call **Backward Pass Differentiable Approximation (BPDA)**.
Many non-differentiable defenses can be expressed as follows:

- Given a pre-trained classifier $f(\cdot)$, construct a preprocessor $g(\cdot)$ and let the secured classifier $\hat{f}(x) = f(g(x))$where the preprocessor $g(\cdot)$ satisfies $g(x) \approx x$ (e.g., such a $g(\cdot)$ may perform image denoising to remove the adversarial perturbation)

Backward Pass Differentiable Approximation (BPDA)

To attack defenses where gradients are not readily available, such as shattered gradients, we introduce a technique we call **Backward Pass Differentiable Approximation (BPDA)**.
Many non-differentiable defenses can be expressed as follows:

- Given a pre-trained classifier $f(\cdot)$, construct a preprocessor $g(\cdot)$ and let the secured classifier $\hat{f}(x) = f(g(x))$where the preprocessor $g(\cdot)$ satisfies $g(x) \approx x$ (e.g., such a $g(\cdot)$ may perform image denoising to remove the adversarial perturbation)
  - If $g(\cdot)$ is smooth and differentiable, then computing gradients through the combined network $\hat{f}$ is often sufficient to circumvent the defense
  - However, recent work has constructed functions $\boldsymbol{g}(\cdot)$ **which are neither smooth nor differentiable**

Backward Pass Differentiable Approximation (BPDA)

To attack defenses where gradients are not readily available, such as shattered gradients, we introduce a technique we call **Backward Pass Differentiable Approximation (BPDA)**.
Many non-differentiable defenses can be expressed as follows:

- Given a pre-trained classifier $f(\cdot)$, construct a preprocessor $g(\cdot)$ and let the secured classifier $\hat{f}(x) = f(g(x))$where the preprocessor $g(\cdot)$ satisfies $g(x) \approx x$ (e.g., such a $g(\cdot)$ may perform image denoising to remove the adversarial perturbation)
  - If $g(\cdot)$ is smooth and differentiable, then computing gradients through the combined network $\hat{f}$ is often sufficient to circumvent the defense
  - However, recent work has constructed functions $\boldsymbol{g}(\cdot)$ **which are neither smooth nor differentiable**
- Because g is constructed with the property that $g(x) \approx x$, **we can approximate its derivative as the derivative of the identity function**: $\nabla_x g(x) \approx \nabla_x x = 1$. Therefore, we can approximate the derivative of $f(g(x))$ at the point $\hat{x}$ as

$$\nabla_x f(g(x))|_{x=\hat{x}} \approx \nabla_x f(x)|_{x=g(\hat{x})}$$

  This allows us to compute gradients and therefore mount a white-box attack.

Backward Pass Differentiable Approximation (BPDA)

To attack defenses where gradients are not readily available, such as shattered gradients, we introduce a technique we call **Backward Pass Differentiable Approximation (BPDA)**.
Many non-differentiable defenses can be expressed as follows:

- Given a pre-trained classifier $f(\cdot)$, construct a preprocessor $g(\cdot)$ and let the secured classifier $\hat{f}(x) = f(g(x))$where the preprocessor $g(\cdot)$ satisfies $g(x) \approx x$ (e.g., such a $g(\cdot)$ may perform image denoising to remove the adversarial perturbation)
  - If $g(\cdot)$ is smooth and differentiable, then computing gradients through the combined network $\hat{f}$ is often sufficient to circumvent the defense
  - However, recent work has constructed functions $\boldsymbol{g}(\cdot)$ **which are neither smooth nor differentiable**
- Because g is constructed with the property that $g(x) \approx x$, **we can approximate its derivative as the derivative of the identity function**: $\nabla_x g(x) \approx \nabla_x x = 1$. Therefore, we can approximate the derivative of $f(g(x))$ at the point $\hat{x}$ as

$$\nabla_x f(g(x))|_{x=\hat{x}} \approx \nabla_x f(x)|_{x=g(\hat{x})}$$

  This allows us to compute gradients and therefore mount a white-box attack.
- We perform **forward propagation** through the neural network **as usual**, but on the **backward pass**, we **replace $g(\cdot)$ with the identity function**.
  - This gives us an approximation of the true gradient.

Expectation over Transformation

For defenses that employ randomized transformations to the input or use a stochastic classifier, we apply Expectation over Transformation (EOT).

- When attacking a classifier $f(\cdot)$ that first randomly transforms its input according to a function $t(\cdot)$ sampled from a distribution of transformations T, **EOT optimizes the expectation over the transformation** $\mathbb{E}_{t\sim T} f(t(x))$.
- The optimization problem can be solved by gradient descent, noting that $\nabla \mathbb{E}_{t\sim T} f(t(x)) = \mathbb{E}_{t\sim T} \nabla f(t(x))$, differentiating through the classifier and transformation, and **approximating the expectation with samples** at each gradient descent step.

Reparameterization

We solve vanishing/exploding gradients by reparameterization.

- Assume we are given a classifier $f(g(x))$ where $g(\cdot)$ performs some optimization loop to transform the input x to a new input $\hat{x}$.
- Often times, this optimization loop means that differentiating through $g(\cdot)$, while possible, yields exploding or vanishing gradients.

Reparameterization

We solve vanishing/exploding gradients by reparameterization.

- Assume we are given a classifier $f(g(x))$ where $g(\cdot)$ performs some optimization loop to transform the input x to a new input $\hat{x}$.
- Often times, this optimization loop means that differentiating through $g(\cdot)$, while possible, yields exploding or vanishing gradients.

To resolve this issue:

- We make a **change-of-variable** $x = h(z)$ for some function $h(\cdot)$ such that $g(h(z)) = h(z)$ for all z, but $h(\cdot)$ is differentiable.
  - For example, if $g(\cdot)$ projects samples to some manifold in a specific manner, we might construct $h(z)$ to return points exclusively on the manifold.
- This allows us to **compute gradients through** $f(h(z))$ and thereby circumvent the defense.

Case Study: ICLR 2018 Defenses

As a case study for evaluating the prevalence of obfuscated gradients, we study the **ICLR 2018** non-certified defenses that argue robustness in a white-box threat model.

- To show a defense can be bypassed, it is only necessary to demonstrate one way to do so; in contrast, a defender must show no attack can succeed.
- **Of the 9 accepted papers, 7 rely on obfuscated gradients.**

Defense	Dataset	Distance	Accuracy
Buckman et al. (2018)	CIFAR	0.031 (ℓ_∞)	0%*
Ma et al. (2018)	CIFAR	0.031 (ℓ_∞)	5%
Guo et al. (2018)	ImageNet	0.005 (ℓ_2)	0%*
Dhillon et al. (2018)	CIFAR	0.031 (ℓ_∞)	0%
Xie et al. (2018)	ImageNet	0.031 (ℓ_∞)	0%*
Song et al. (2018)	CIFAR	0.031 (ℓ_∞)	9%*
Samangouei et al. (2018)	MNIST	0.005 (ℓ_2)	55%**
Madry et al. (2018)	CIFAR	0.031 (ℓ_∞)	47%
Na et al. (2018)	CIFAR	0.015 (ℓ_∞)	15%

Table 1. **Summary of Results:** Seven of nine defense techniques accepted at ICLR 2018 cause obfuscated gradients and are vulnerable to our attacks. Defenses denoted with $*$ propose combining adversarial training; we report here the defense alone, see §5 for full numbers. The fundamental principle behind the defense denoted with $**$ has 0% accuracy; in practice, imperfections cause the theoretically optimal attack to fail, see §5.4.2 for details.

Adversarial Training

Adversarial training does not cause obfuscated gradients and it **passes all tests** for characteristic behaviors of obfuscated gradients that we list. However it has some limitation

- Adversarial retraining has been shown to be **difficult at ImageNet scale**
- Training exclusively on ℓ_∞ adversarial examples provides only **limited robustness** to adversarial examples under **other distortion metrics**

Example 1: Thermometer Encoding

Defense Details are provided by Buckman et al. [2]

- Given an image x, for each pixel color $x_{i,j,c}$, the l-level thermometer encoding $\tau(x_{i,j,c})$ is a l-dimensional vector where $\tau(x_{i,j,c})_k = 1$ if $x_{i,j,c} > \frac{k}{l}$, and 0 otherwise.
  - For a 10-level thermometer encoding, $\tau(0.66) = 1111110000$.

Example 1: Thermometer Encoding

Defense Details are provided by Buckman et al. [2]

- Given an image x, for each pixel color $x_{i,j,c}$, the l-level thermometer encoding $\tau(x_{i,j,c})$ is a l-dimensional vector where $\tau(x_{i,j,c})_k = 1$ if $x_{i,j,c} > \frac{k}{l}$, and 0 otherwise.
  - For a 10-level thermometer encoding, $\tau(0.66) = 1111110000$.

Discussion

- This defense causes **gradient shattering**.
- This can be observed through **black-box attack evaluation**:
  - Adversarial examples generated on a standard adversarially trained model transfer to a thermometer encoded model reducing the accuracy to 67%, well below the 80% robustness to the white-box iterative attack.

Example 1: Thermometer Encoding

Defense Details are provided by Buckman et al. [2]

- Given an image x, for each pixel color $x_{i,j,c}$, the l-level thermometer encoding $\tau(x_{i,j,c})$ is a l-dimensional vector where $\tau(x_{i,j,c})_k = 1$ if $x_{i,j,c} > \frac{k}{l}$, and 0 otherwise.
  - For a 10-level thermometer encoding, $\tau(0.66) = 1111110000$.

Discussion

- This defense causes **gradient shattering**.
- This can be observed through **black-box attack evaluation**:
  - Adversarial examples generated on a standard adversarially trained model transfer to a thermometer encoded model reducing the accuracy to 67%, well below the 80% robustness to the white-box iterative attack.

Evaluation

- We use the **BPDA** approach where we let $f(x) = \tau(x)$. Observe that if we define:

$$\hat{\tau}(x_{i,j,c})_k = \min\left(\max\left(\frac{x_{i,j,c}}{k/l}, 0\right), 1\right)$$

then:

$$\tau(x_{i,j,c})_k = \lfloor \hat{\tau}(x_{i,j,c})_k \rfloor$$

So we can let $g(x) = \hat{\tau}(x)$ and replace the backwards pass with the function $g(\cdot)$.

Example 1: Thermometer Encoding

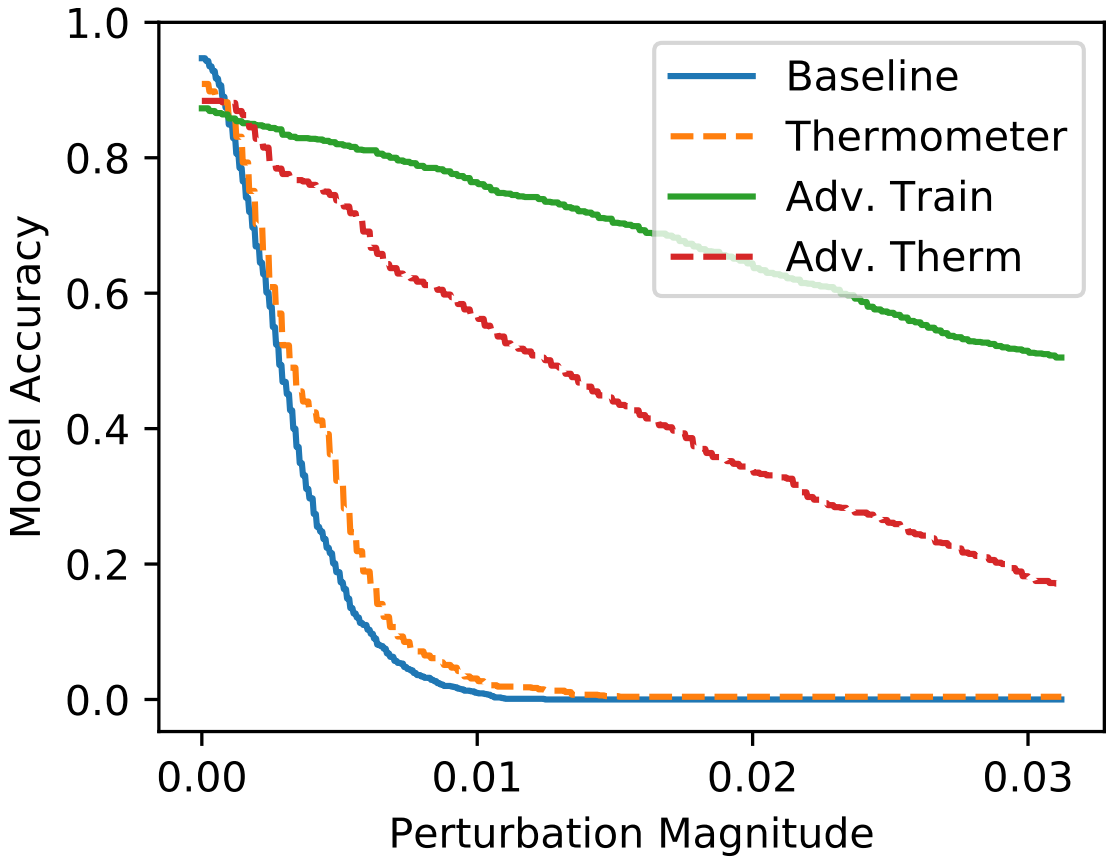


Figure: Model accuracy versus distortion (under ℓ_∞). Adversarial training increases robustness to 50% at $\epsilon = 8/255$; thermometer encoding by itself provides limited value, and when coupled with adversarial training performs worse than adversarial training alone.

Example 2: Input Transformation

Guo et al. [6] propose five input transformations to counter adversarial examples:

1. Image cropping and rescaling
2. Bit-depth reduction
3. JPEG compression
4. Total variance minimization
5. Image quilting (reconstruct images by replacing small patches with patches from clean images)

Example 2: Input Transformation

Guo et al. [6] propose five input transformations to counter adversarial examples:

1. Image cropping and rescaling
2. Bit-depth reduction
3. JPEG compression
4. Total variance minimization
5. Image quilting (reconstruct images by replacing small patches with patches from clean images)

To overcome transformations:

- We circumvent **image cropping and rescaling** with a direct application of **EOT**.
- To circumvent **bit-depth reduction and JPEG compression**, we use BPDA and approximate the **backward pass with the identity function**.
- To circumvent **total variance minimization and image quilting**, which are both non-differentiable and randomized, we apply **EOT** and use **BPDA** to approximate the gradient through the transformation.

Example 3: Stochastic Gradients

- **Stochastic Activation Pruning (SAP)**
  - SAP [4] **randomly drops some neurons** of each layer f^i to 0 with probability proportional to their absolute value.
- **Mitigating Through Randomization**
  - Xie et al. [11] propose to defend against adversarial examples by adding a randomization layer before the input to the classifier.
    - It **randomly rescales** the image and add zero-pad to it.

Example 3: Stochastic Gradients

- **Stochastic Activation Pruning (SAP)**
  - SAP [4] **randomly drops some neurons** of each layer f^i to 0 with probability proportional to their absolute value.
- **Mitigating Through Randomization**
  - Xie et al. [11] propose to defend against adversarial examples by adding a randomization layer before the input to the classifier.
    - It **randomly rescales** the image and add zero-pad to it.

To circumvent both defenses, we estimate the gradients by computing the expectation over instantiations of randomness.

- At each iteration of gradient descent, instead of taking a step in the direction of $\nabla_x f(x)$ we move in the direction of $\sum_{i=1}^{k} \nabla_x f(x)$.

Example 4: PixelDefend

Song et al. [10] propose using a PixelCNN generative model to project a potential adversarial example back onto the data manifold before feeding it into a classifier.

- PixelDefend **purifies** adversarially perturbed images prior to classification by using a greedy decoding procedure to approximate finding the highest probability example within an ϵ-ball of the input image.

Example 4: PixelDefend

Song et al. [10] propose using a PixelCNN generative model to project a potential adversarial example back onto the data manifold before feeding it into a classifier.

- PixelDefend **purifies** adversarially perturbed images prior to classification by using a greedy decoding procedure to approximate finding the highest probability example within an ϵ-ball of the input image.

To circumventing PixelDefend:

- We sidestep the problem of computing gradients through an unrolled version of PixelDefend by approximating gradients with BPDA (**approximate PixelCNN derivative as the derivative of the identity function**).

References and Resources I

- Adversarial Training (AT) was first introduced by Goodfellow et al. [5], however the adversary utilized was quite weak. Madry et al. [8] were the first to use the PGD attack to improve their results. As a consequence of their wildly successful paper, their method of Adversarial Training is now considered to be the standard version.
- Athalye, Carlini, and Wagner [1] identified obfuscated gradients and broke 7 out of 9 defenses [2, 6, 4, 11, 10] from ICLR 2018. Only Adversarial Training [8] managed to retain its performance and as a result, emerged as the most widely used defense mechanism against adversarial examples.
- Zhang et al. [12] identify a trade-off between robustness and accuracy that serves as a guiding principle in the design of defenses against adversarial examples.
- Currently, different variants of Adversarial Training dominate Robust Bench [3].

References and Resources II

[1] Anish Athalye, Nicholas Carlini, and David Wagner. **Obfuscated Gradients Give a False Sense of Security: Circumventing Defenses to Adversarial Examples**, 2018. URL https://arxiv.org/abs/1802.00420.

[2] Jacob Buckman, Aurko Roy, Colin Raffel, and Ian Goodfellow. Thermometer Encoding: One Hot Way to Resist Adversarial Examples. In **International Conference on Learning Representations**, 2018. URL https://openreview.net/forum?id=S18Su--CW.

[3] Francesco Croce, Maksym Andriushchenko, Vikash Sehwag, Edoardo Debenedetti, Nicolas Flammarion, Mung Chiang, Prateek Mittal, and Matthias Hein. RobustBench: A Standardized Adversarial Robustness Benchmark, 2021. URL https://arxiv.org/abs/2010.09670.

[4] Guneet S. Dhillon, Kamyar Azizzadenesheli, Zachary C. Lipton, Jeremy Bernstein, Jean Kossaifi, Aran Khanna, and Anima Anandkumar. Stochastic activation pruning for robust adversarial defense, 2018. URL https://arxiv.org/abs/1803.01442.

[5] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and Harnessing Adversarial Examples, 2015. URL https://arxiv.org/abs/1412.6572.

References and Resources III

[6] Chuan Guo, Mayank Rana, Moustapha Cisse, and Laurens van der Maaten. Countering Adversarial Images using Input Transformations, 2018. URL https://arxiv.org/abs/1711.00117.

[7] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the Knowledge in a Neural Network, 2015. URL https://arxiv.org/abs/1503.02531.

[8] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. **Towards Deep Learning Models Resistant to Adversarial Attacks**, 2019. URL https://arxiv.org/abs/1706.06083.

[9] Nicolas Papernot, Patrick McDaniel, Xi Wu, Somesh Jha, and Ananthram Swami. Distillation as a Defense to Adversarial Perturbations against Deep Neural Networks, 2016. URL https://arxiv.org/abs/1511.04508.

[10] Yang Song, Taesup Kim, Sebastian Nowozin, Stefano Ermon, and Nate Kushman. PixelDefend: Leveraging Generative Models to Understand and Defend against Adversarial Examples, 2018. URL https://arxiv.org/abs/1710.10766.

[11] Cihang Xie, Jianyu Wang, Zhishuai Zhang, Zhou Ren, and Alan Yuille. Mitigating Adversarial Effects Through Randomization, 2018. URL https://arxiv.org/abs/1711.01991.

[12] Hongyang Zhang, Yaodong Yu, Jiantao Jiao, Eric P. Xing, Laurent El Ghaoui, and Michael I. Jordan. **Theoretically Principled Trade-off between Robustness and Accuracy**, 2019. URL https://arxiv.org/abs/1901.08573.